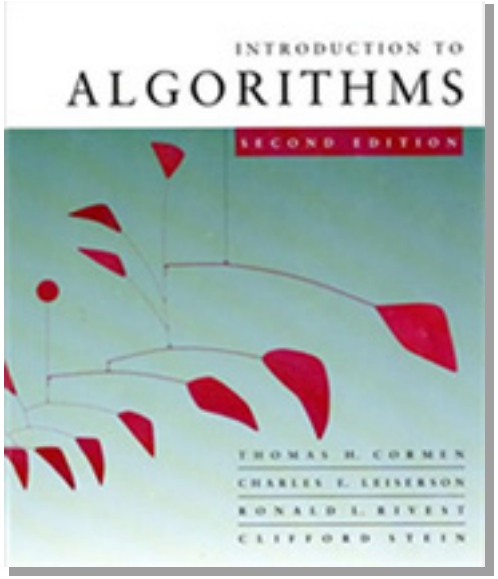
Introduction to Algorithms

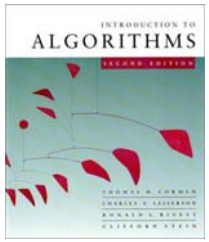


LECTURE 12

Dynamic Programming

- Longest common subsequence
- Optimal substructure
- Overlapping subproblems

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



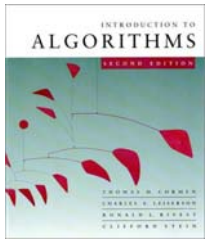
Dynamic programming

Design technique, like divide-and-conquer.

Example: *Longest Common Subsequence (LCS)*

- Given two sequences $x[1 \ldots m]$ and $y[1 \ldots n]$, find a longest subsequence common to them both.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Dynamic programming

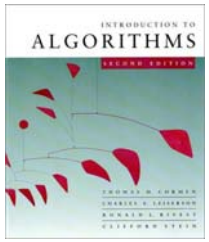
Design technique, like divide-and-conquer.

Example: *Longest Common Subsequence (LCS)*

- Given two sequences $x[1 \,.\,.\, m]$ and $y[1 \,.\,.\, n]$, find a longest subsequence common to them both.

"a" *not* "the"

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Dynamic programming

Design technique, like divide-and-conquer.

Example: *Longest Common Subsequence (LCS)*

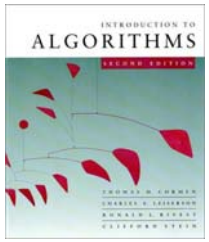
- Given two sequences $x[1 \,.\,.\, m]$ and $y[1 \,.\,.\, n]$, find a longest subsequence common to them both.

"a" *not* "the"

x: A B C B D A B

y: B D C A B A

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Dynamic programming

Design technique, like divide-and-conquer.

Example: *Longest Common Subsequence (LCS)*

- Given two sequences $x[1 \,.\,.\, m]$ and $y[1 \,.\,.\, n]$, find a longest subsequence common to them both.

"a" *not* "the"

x:	A	B	C	B	D	A	B
y:	B	D	C	A	B	A	

BCBA = LCS(x, y)

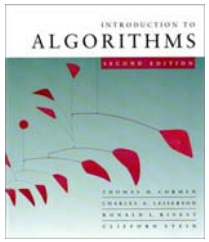
functional notation, but not a function

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Brute-force LCS algorithm

Check every subsequence of $x[1 \mathinner{..} m]$ to see if it is also a subsequence of $y[1 \mathinner{..} n]$.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Brute-force LCS algorithm

Check every subsequence of $x[1 \mathinner{..} m]$ to see if it is also a subsequence of $y[1 \mathinner{..} n]$.

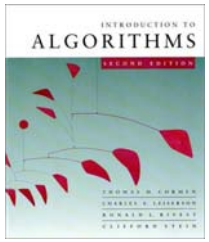
Analysis

- Checking $= O(n)$ time per subsequence.
- 2^m subsequences of x (each bit-vector of length m determines a distinct subsequence of x).

Worst-case running time $= O(n2^m)$

$=$ exponential time.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

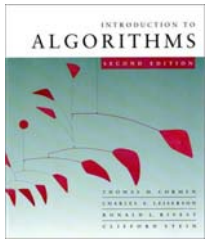


Towards a better algorithm

Simplification:

1. Look at the *length* of a longest-common subsequence.
2. Extend the algorithm to find the LCS itself.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



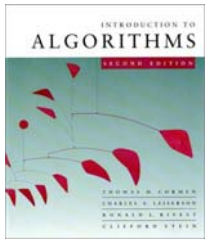
Towards a better algorithm

Simplification:

1. Look at the *length* of a longest-common subsequence.
2. Extend the algorithm to find the LCS itself.

Notation: Denote the length of a sequence s by $|s|$.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Towards a better algorithm

Simplification:

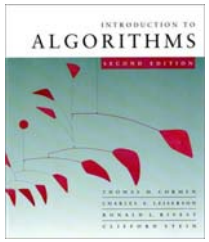
1. Look at the *length* of a longest-common subsequence.
2. Extend the algorithm to find the LCS itself.

Notation: Denote the length of a sequence s by $|s|$.

Strategy: Consider ***prefixes*** of x and y.

- Define $c[i, j] = |\text{LCS}(x[1 \,..\, i], y[1 \,..\, j])|$.
- Then, $c[m, n] = |\text{LCS}(x, y)|$.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Recursive formulation

Theorem.

$$c[i, j] = \begin{cases} c[i-1, j-1] + 1 & \text{if } x[i] = y[j], \\ \max\{c[i-1, j], c[i, j-1]\} & \text{otherwise.} \end{cases}$$

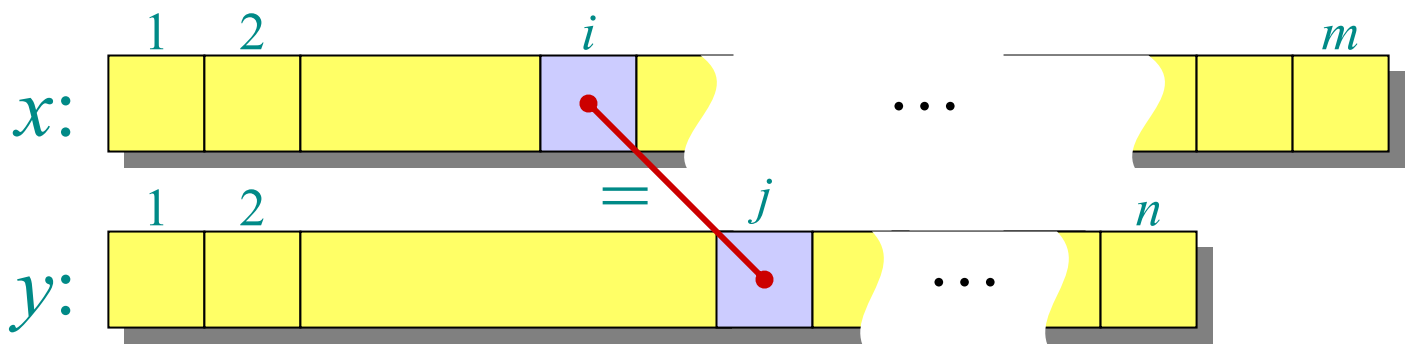
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Recursive formulation

Theorem.

$$c[i,j] = \begin{cases} c[i-1,j-1] + 1 & \text{if } x[i] = y[j], \\ \max\{c[i-1,j], c[i,j-1]\} & \text{otherwise.} \end{cases}$$

Proof. Case $x[i] = y[j]$:



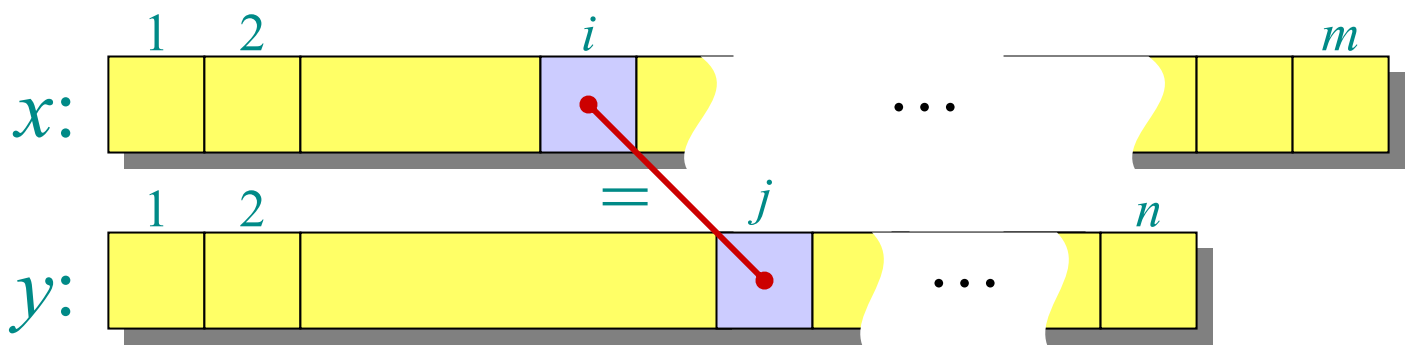
Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Recursive formulation

Theorem.

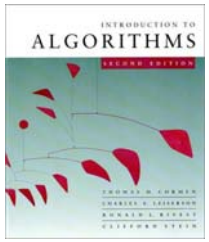
$$c[i, j] = \begin{cases} c[i-1, j-1] + 1 & \text{if } x[i] = y[j], \\ \max\{c[i-1, j], c[i, j-1]\} & \text{otherwise.} \end{cases}$$

Proof. Case $x[i] = y[j]$:



Let $z[1 \mathinner{..} k] = \text{LCS}(x[1 \mathinner{..} i], y[1 \mathinner{..} j])$, where $c[i, j] = k$. Then, $z[k] = x[i]$, or else z could be extended. Thus, $z[1 \mathinner{..} k-1]$ is CS of $x[1 \mathinner{..} i-1]$ and $y[1 \mathinner{..} j-1]$.

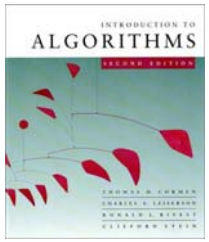
Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Proof (continued)

Claim: $z[1 \,..\, k-1] = \text{LCS}(x[1 \,..\, i-1], y[1 \,..\, j-1])$. Suppose w is a longer CS of $x[1 \,..\, i-1]$ and $y[1 \,..\, j-1]$, that is, $|w| > k-1$. Then, ***cut and paste***: $w \,\|\, z[k]$ (w concatenated with $z[k]$) is a common subsequence of $x[1 \,..\, i]$ and $y[1 \,..\, j]$ with $|w \,\|\, z[k]| > k$. Contradiction, proving the claim.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Proof (continued)

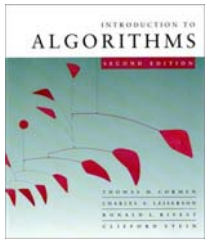
Claim: $z[1 \,..\, k\text{–}1] = \text{LCS}(x[1 \,..\, i\text{–}1], y[1 \,..\, j\text{–}1])$.

Suppose w is a longer CS of $x[1 \,..\, i\text{–}1]$ and $y[1 \,..\, j\text{–}1]$, that is, $|w| > k\text{–}1$. Then, ***cut and paste***: $w \,\|\, z[k]$ (w concatenated with $z[k]$) is a common subsequence of $x[1 \,..\, i]$ and $y[1 \,..\, j]$ with $|w \,\|\, z[k]| > k$. Contradiction, proving the claim.

Thus, $c[i\text{–}1, j\text{–}1] = k\text{–}1$, which implies that $c[i, j] = c[i\text{–}1, j\text{–}1] + 1$.

Other cases are similar. ☐

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

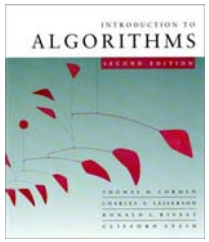


Dynamic-programming hallmark #1

Optimal substructure

An optimal solution to a problem (instance) contains optimal solutions to subproblems.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



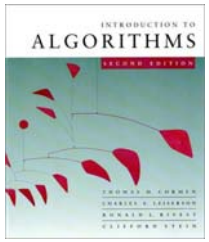
Dynamic-programming hallmark #1

Optimal substructure

An optimal solution to a problem (instance) contains optimal solutions to subproblems.

If $z = \mathrm{LCS}(x, y)$, then any prefix of z is an LCS of a prefix of x and a prefix of y.

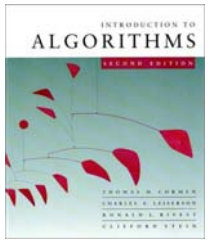
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Recursive algorithm for LCS

$$
\begin{aligned}
&\text{LCS}(x, y, i, j) \\
&\quad \textbf{if } x[i] = y[j] \\
&\qquad \textbf{then } c[i, j] \leftarrow \text{LCS}(x, y, i-1, j-1) + 1 \\
&\qquad \textbf{else } c[i, j] \leftarrow \max\{\text{LCS}(x, y, i-1, j), \\
&\qquad\qquad\qquad\qquad\qquad\qquad \text{LCS}(x, y, i, j-1)\}
\end{aligned}
$$

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Recursive algorithm for LCS

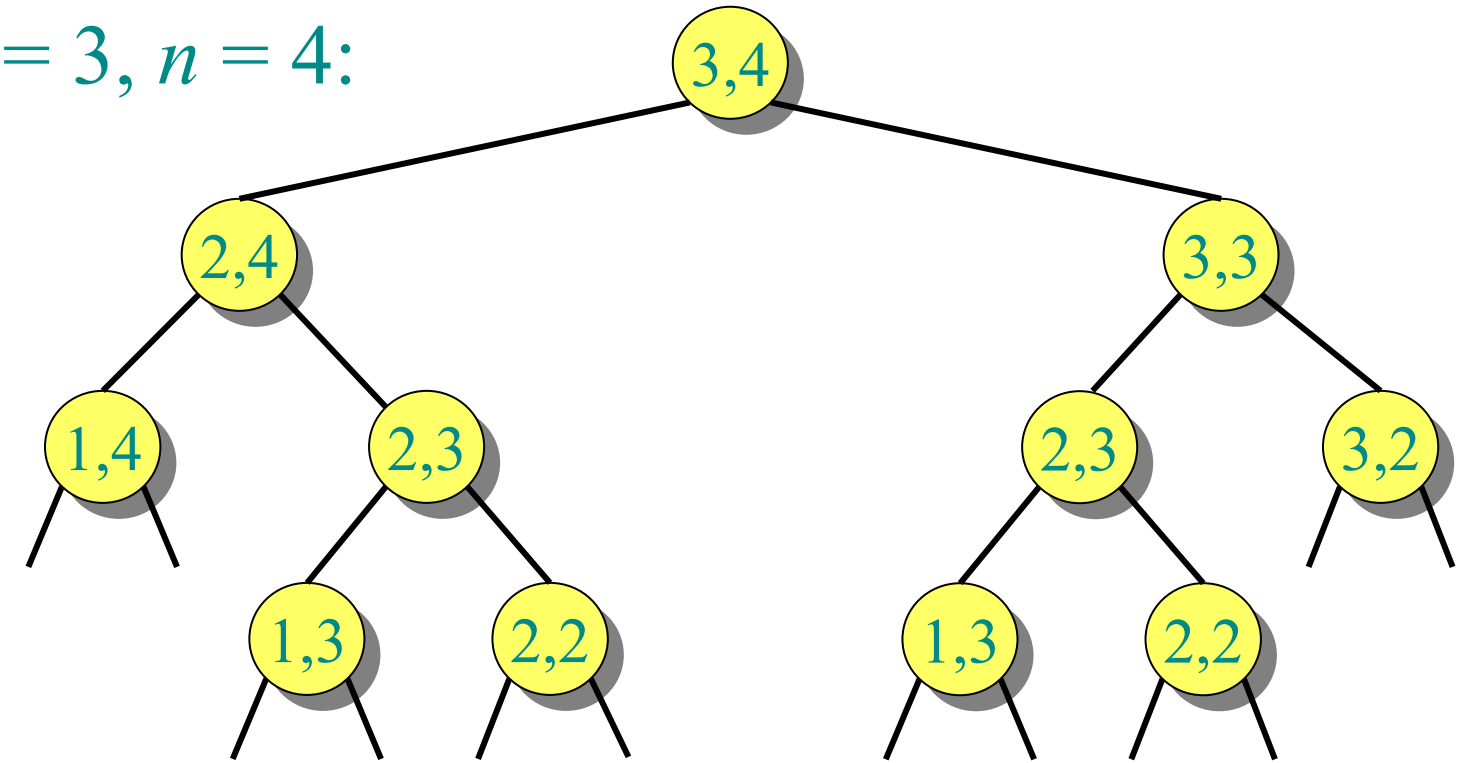
LCS(x, y, i, j)
  if $x[i] = y[j]$
    then $c[i, j] \leftarrow$ LCS$(x, y, i-1, j-1) + 1$
    else $c[i, j] \leftarrow \max\{LCS(x, y, i-1, j)$,
      LCS$(x, y, i, j-1)\}$

Worst-case: $x[i] \neq y[j]$, in which case the algorithm evaluates two subproblems, each with only one parameter decremented.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Recursion tree

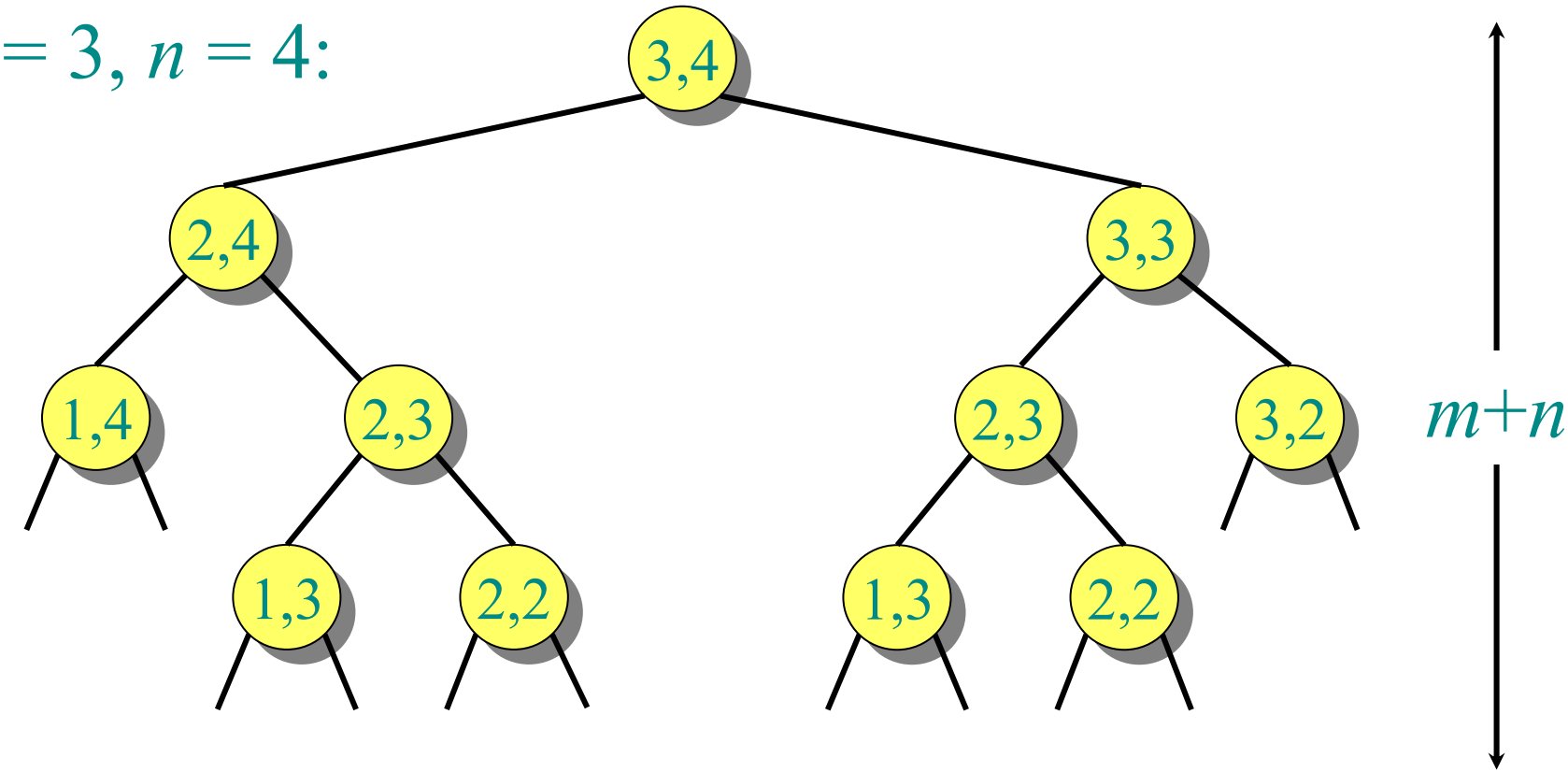
$m = 3$, $n = 4$:



 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Recursion tree

$m = 3$, $n = 4$:

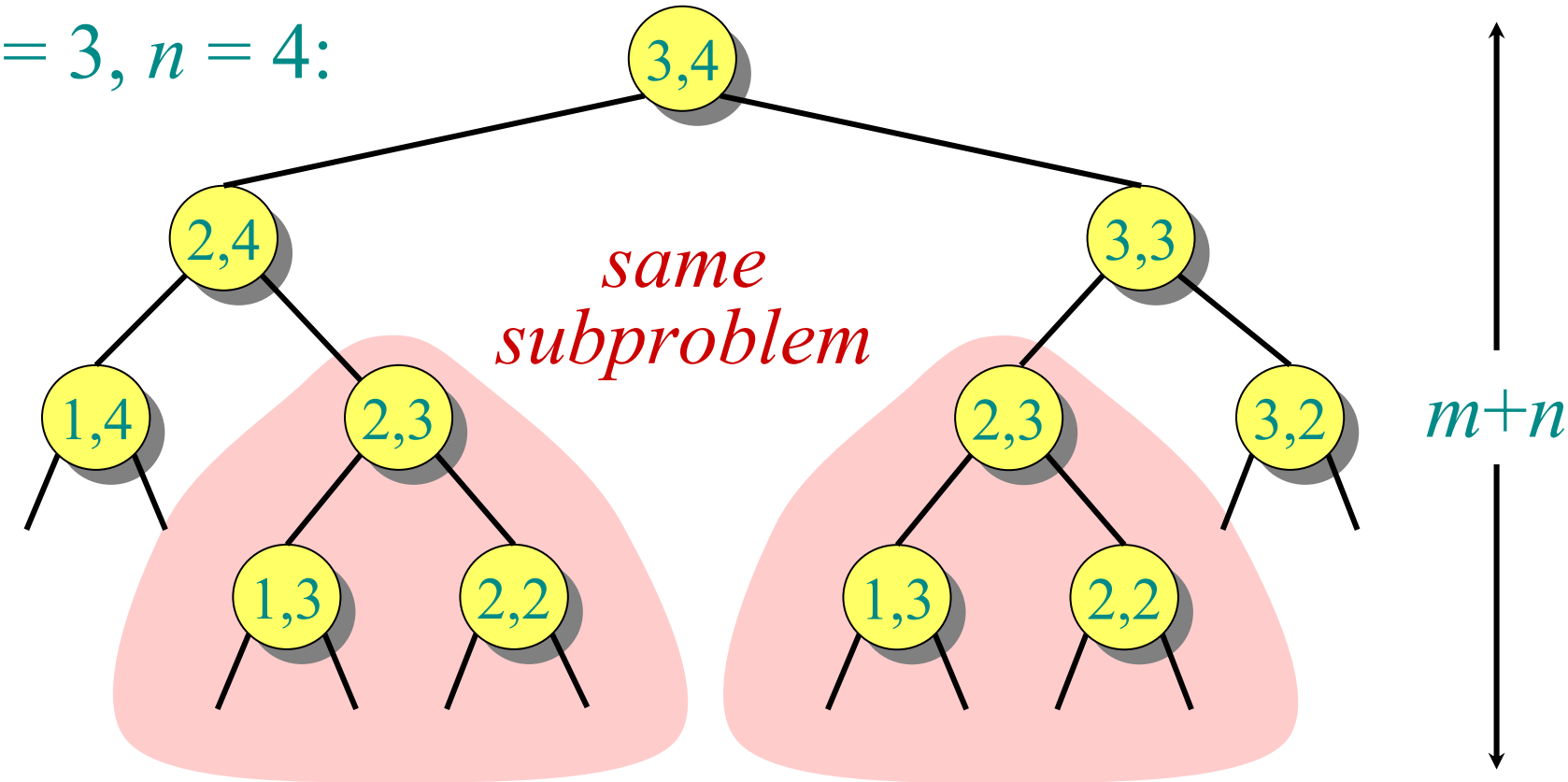


Height $= m + n \Rightarrow$ work potentially exponential.

Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

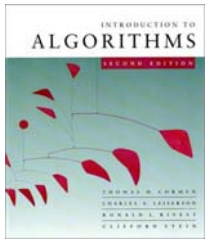
Recursion tree

$m = 3$, $n = 4$:



Height $= m + n \Rightarrow$ work potentially exponential, but we're solving subproblems already solved!

Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

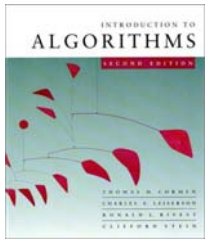


Dynamic-programming hallmark #2

Overlapping subproblems

A recursive solution contains a "small" number of distinct subproblems repeated many times.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



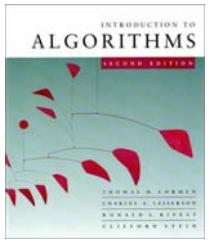
Dynamic-programming hallmark #2

Overlapping subproblems

A recursive solution contains a "small" number of distinct subproblems repeated many times.

The number of distinct LCS subproblems for two strings of lengths m and n is only mn.

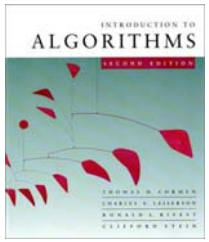
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Memoization algorithm

Memoization: After computing a solution to a subproblem, store it in a table. Subsequent calls check the table to avoid redoing work.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



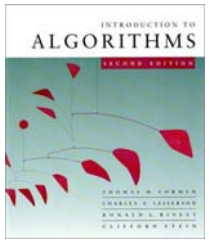
Memoization algorithm

Memoization: After computing a solution to a subproblem, store it in a table. Subsequent calls check the table to avoid redoing work.

LCS(x, y, i, j)
  if $c[i, j] = \text{NIL}$
    then if $x[i] = y[j]$
      then $c[i, j] \leftarrow \text{LCS}(x, y, i-1, j-1) + 1$
      else $c[i, j] \leftarrow \max\{\text{LCS}(x, y, i-1, j),$
        $\text{LCS}(x, y, i, j-1)\}$

same as before

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Memoization algorithm

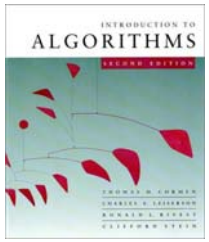
Memoization: After computing a solution to a subproblem, store it in a table. Subsequent calls check the table to avoid redoing work.

LCS(x, y, i, j)
  if $c[i, j] = \text{NIL}$
    then if $x[i] = y[j]$
      then $c[i, j] \leftarrow \text{LCS}(x, y, i-1, j-1) + 1$
      else $c[i, j] \leftarrow \max\{\text{LCS}(x, y, i-1, j),$
        $\text{LCS}(x, y, i, j-1)\}$

same as before

Time $= \Theta(mn) =$ constant work per table entry.
Space $= \Theta(mn)$.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



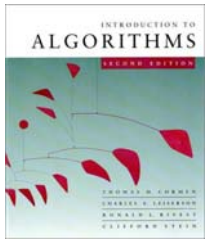
Dynamic-programming algorithm

IDEA:

Compute the table bottom-up.

		A	B	C	B	D	A	B
	0	0	0	0	0	0	0	0
B	0	0	1	1	1	1	1	1
D	0	0	1	1	1	2	2	2
C	0	0	1	2	2	2	2	2
A	0	1	1	2	2	2	3	3
B	0	1	2	2	3	3	3	4
A	0	1	2	2	3	3	4	4

Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Dynamic-programming algorithm

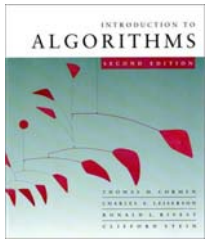
IDEA:

Compute the table bottom-up.

Time $= \Theta(mn)$.

		A	B	C	B	D	A	B
	0	0	0	0	0	0	0	0
B	0	0	1	1	1	1	1	1
D	0	0	1	1	1	2	2	2
C	0	0	1	2	2	2	2	2
A	0	1	1	2	2	2	3	3
B	0	1	2	2	3	3	3	4
A	0	1	2	2	3	3	4	4

Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Dynamic-programming algorithm

IDEA:

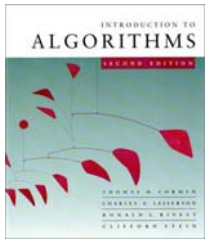
Compute the table bottom-up.

Time $= \Theta(mn)$.

Reconstruct LCS by tracing backwards.

		A	B	C	B	D	A	B
	0	0	0	0	0	0	0	0
B	0	0	1	1	1	1	1	1
D	0	0	1	1	1	2	2	2
C	0	0	1	2	2	2	2	2
A	0	1	1	2	2	2	3	3
B	0	1	2	2	3	3	3	4
A	0	1	2	2	3	3	4	4

Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson



Dynamic-programming algorithm

IDEA:

Compute the table bottom-up.

Time = $\Theta(mn)$.

Reconstruct LCS by tracing backwards.

Space = $\Theta(mn)$.

Exercise:

$O(\min\{m, n\})$.

		A	B	C	B	D	A	B
	0	0	0	0	0	0	0	0
B	0	0	1	1	1	1	1	1
D	0	0	1	1	1	2	2	2
C	0	0	1	2	2	2	2	2
A	0	1	1	2	2	2	3	3
B	0	1	2	2	3	3	3	4
A	0	1	2	2	3	3	4	4

Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson